# RTL Implementation of the KCF/MOSSE Visual Tracking Algorithm on FPGA

*Project report submitted in partial fulfillment of the requirement for the degree of*

Bachelor of Technology
in
Electrical and Computer Engineering

Submitted by

Akhil Sriram (Roll No. [2210110134])

Under Supervision of

Dr. Venkatnarayan Hariharan
Department of Electrical Engineering

SHIV NADAR
INSTITUTION OF EMINENCE DEEMED TO BE UNIVERSITY
DELHI NCR

SCHOOL OF ENGINEERING

**Department of Electrical Engineering**
**School of Engineering**
**Shiv Nadar Institution of Eminence**

(April 2026)

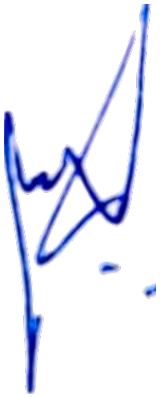
# Candidate Declaration

I hereby declare that the thesis entitled "KCF/MOSSE Visual Tracking Algorithm on FPGA" submitted for the B. Tech. degree program. This thesis has been written in my own words. I have adequately cited and referenced the original sources.

(Signature)
Akhil Sriram
(2210110134)

# Certificate

It is certified that the work contained in the project report titled "RTL Design of a KCF Visual Tracker," by **Akhil Sriram** has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.



(Signature)
Dr. Venkatnarayan Hariharan
Dept. of Electrical Engineering
School of Engineering
Shiv Nadar Institution of Eminence
Date: _____

# Abstract

Visual object tracking is a critical task in computer vision, but traditional software implementations on general-purpose processors often suffer from high latency and power consumption. This project presents a purely hardware-based implementation of the MOSSE (Minimum Output Sum of Squared Error) correlation filter tracking algorithm, designed entirely in Verilog HDL. Unlike software solutions that run sequentially on a CPU, this project leverages the parallel processing capabilities of an FPGA (Field-Programmable Gate Array) to perform high-speed tracking. The design focuses on translating the mathematical operations of the MOSSE algorithm—specifically the Fast Fourier Transforms (FFT) and complex conjugate multiplications—into dedicated digital logic circuits (RTL). The system is designed to operate as a standalone IP core. It accepts image data (such as pixel streams or memory initialization files) as input, processes the frames using a pipeline of custom hardware modules, and outputs the target coordinates. The implementation will be verified through behavioral simulation and synthesized on an FPGA to demonstrate the efficiency and speed advantages of a dedicated hardware solution compared to software-based execution.

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

Visual object tracking is the task of continuously estimating the position of a target object across a sequence of video frames, given only its initial location. It is a critical building block for applications including autonomous vehicles, unmanned aerial vehicles (UAVs), surveillance systems, and human-computer interaction. The core challenge lies in handling real-world variations such as illumination changes, partial occlusion, scale variation, and background clutter, while maintaining real-time throughput.

General-purpose processors (CPUs) execute tracking algorithms sequentially and are limited by memory bandwidth and clock frequency when processing high-resolution video streams. Field-Programmable Gate Arrays (FPGAs), by contrast, offer massively parallel dataflow architectures, dedicated DSP blocks, and configurable on-chip memory that can be tailored precisely to the arithmetic structure of a given algorithm. This project exploits these properties to implement the KCF tracker as a dedicated hardware IP core.

## 1.1 Background and Motivation

The demand for real-time tracking in embedded systems has grown significantly. Deployments on edge devices — cameras mounted on drones, robots, or smart infrastructure — impose strict power and latency budgets that general-purpose compute cannot meet. Dedicated FPGA accelerators can achieve an order-of-magnitude improvement in throughput-per-watt compared to equivalent CPU implementations, while remaining reconfigurable unlike ASICs.

The Kernelized Correlation Filter [1] is a particularly strong candidate for hardware acceleration because its dominant operations — the 2D-FFT, element-wise complex multiplication, and 2D-IFFT — map directly to well-understood hardware primitives. All arithmetic is bounded and predictable, making it amenable to fixed-point implementation without requiring floating-point units.

## 1.2 The KCF Approach

The KCF tracker operates in the frequency domain by treating target patches as cyclic shifts of a base sample. The key insight from Henriques et al. [1] is that a matrix of all cyclic shifts of a vector is a *circulant matrix*, and any circulant matrix is diagonalised by

the Discrete Fourier Transform (DFT). This means that ridge regression over all cyclic-shift augmented samples reduces to a simple element-wise division in the frequency domain:

$$\hat{\alpha} = \frac{\hat{Y}}{|\hat{X}|^2 + \lambda} \tag{1.1}$$

where $\hat{X} = \mathcal{F}(x)$ is the FFT of the windowed training patch, $\hat{Y} = \mathcal{F}(y)$ is the FFT of the desired Gaussian response label, and $\lambda$ is a regularisation constant. The total computational complexity is $\mathcal{O}(N^2 \log N)$ per frame, compared to $\mathcal{O}(N^6)$ for a naive kernel ridge regression.

## 1.3 Project Scope

This project implements the KCF tracking pipeline as a synthesisable SystemVerilog IP core with the following parameters and constraints:

- Patch size: $N = 32$ pixels ($32 \times 32 = 1024$ pixels per patch)
- Kernel: linear (dot-product kernel; equivalent to standard ridge regression in feature space)
- Arithmetic: Q8.8 signed fixed-point (16-bit, 8 fractional bits)
- Training: offline — filter coefficients $\hat{\alpha}$ pre-computed in Python and loaded via `$readmemh` into on-chip ROM
- Detection: online — per-frame pipeline processing live patch data
- Interface: AXI4-Stream (pixel ingestion) + AXI4-Lite (control and result registers)
- Target hardware: Xilinx Artix-7 XC7A100T (Nexys A7-100T board)

# Chapter 2

# Literature Survey

This chapter surveys the key algorithmic and hardware prior works that directly inform the design decisions made in this project.

## 2.1 MOSSE Filter — Bolme et al., 2010

The Minimum Output Sum of Squared Error (MOSSE) filter [2] was the first correlation-filter-based tracker to demonstrate real-time performance (669 fps). Given a training patch $f$ and desired response $g$, the optimal frequency-domain filter $H^*$ is:

$$H^* = \frac{\sum_i \hat{G}_i \odot \overline{\hat{F}_i}}{\sum_i \overline{\hat{F}_i} \odot \hat{F}_i} \tag{2.1}$$

where $\hat{G}_i$ and $\hat{F}_i$ are the FFTs of the desired response and training patch respectively, $\overline{(\cdot)}$ denotes complex conjugation, and $\odot$ denotes element-wise multiplication. MOSSE uses a single feature channel (raw grayscale pixels) and a squared loss, which makes it sensitive to illumination changes and appearance variation. The KCF generalises MOSSE by introducing a kernel mapping and multiple feature channels.

## 2.2 KCF Tracker — Henriques et al., 2015

The KCF tracker [1] extends the correlation filter framework to the kernelised (non-linear) setting using the kernel trick. For a Radial Basis Function (RBF) or Gaussian kernel, the dual-space filter coefficients in the frequency domain satisfy:

$$\hat{\alpha} = \frac{\hat{Y}}{\hat{k}^{xx} + \lambda} \tag{2.2}$$

where $\hat{k}^{xx}$ is the FFT of the kernel correlation between the training patch and itself. For the special case of the linear kernel, $\hat{k}^{xx} = |\hat{X}|^2$ element-wise, which is the formula implemented in this project (see Equation 1.1).

A key property exploited by KCF is that the circulant structure of the training data allows the kernel matrix to be diagonalised by the DFT, reducing the $\mathcal{O}(N^4)$ kernel

evaluation to $\mathcal{O}(N^2 \log N)$ per frame. This makes real-time operation feasible even on relatively modest hardware.

## 2.3 Normalised Cross-Correlation (NCC) for Target Acquisition

Normalised Cross-Correlation (NCC) is a classical template-matching technique used in this project for the initial target acquisition step. Given a template $T$ of size $M \times M$ and a search image $I$ of size $P \times P$, the NCC score at position $(r, c)$ is:

$$\text{NCC}(r, c) = \frac{\sum_{i,j} T(i, j) \cdot I(r + i, c + j)}{\sqrt{\sum_{i,j} T(i, j)^2 \cdot \sum_{i,j} I(r + i, c + j)^2}} \tag{2.3}$$

This can be computed efficiently in $\mathcal{O}(PN \log(PN))$ using the FFT convolution theorem: $\text{corr}(T, I) = \mathcal{F}^{-1}(\overline{\hat{T}} \odot \hat{I})$. In this project, NCC is applied once to locate the target in Frame 2 before handing control to the KCF tracking loop.

## 2.4 Hardware FFT — Cooley-Tukey Radix-2 DIT

The Cooley-Tukey algorithm [3] decomposes an $N$-point DFT into $\log_2 N$ stages of butterfly computations, reducing complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \log_2 N)$. The 2D-FFT is computed by applying the 1D-FFT row-wise and then column-wise (separable row-column decomposition). For $N = 32$, this requires 5 butterfly stages, each accessing precomputed twiddle factor ROMs.

Hardware FFT implementations for FPGAs typically use the butterfly unit as the core computation element, with pipelined or iterative architectures depending on the area-throughput tradeoff. This project uses an iterative architecture (`fft1d_32.sv`) that computes one butterfly per clock cycle, processing each 32-point FFT in 80 cycles.

## 2.5 Algorithm Comparison

Table 2.1 summarises the three tracking approaches relevant to this project.

Table 2.1: Comparison of Tracking Algorithms

| **Property** | **NCC** | **MOSSE** | **KCF** |
|---|---|---|---|
| Purpose | Acquisition | Tracking | Tracking |
| Feature Space | Raw pixels | Raw pixels | Linear / RBF kernel |
| Complexity | $O(MN \log MN)$ | $O(N^2 \log N)$ | $O(N^2 \log N)$ |
| Robustness | Low | Medium | High |
| Update Rule | None | Online average | Learning-rate $\eta$ |
| HW Cost | Medium (1 FFT) | Low (2 FFT) | Medium (6 FFT) |
| Kernel | None | Linear | Configurable |
| This Project | Frame 2 only | — | Frames 3+ |

# Chapter 3

# Research and Implementation

## 3.1 KCF Algorithm Implementation

### 3.1.1 Training Phase

Given a $32 \times 32$ training patch $x$ extracted from the first video frame, the offline training procedure computes the KCF filter coefficients $\hat{\alpha}$. The steps are:

1. **Hann Windowing:** Multiply $x$ element-wise by a 2D Hann window to suppress spectral leakage at patch boundaries:

$$w[n] = 0.5 - 0.5\cos\left(\frac{2\pi n}{N-1}\right), \quad x_w = x \odot (w \otimes w) \tag{3.1}$$

2. **2D-FFT:** Compute $\hat{X} = \mathcal{F}_{\text{2D}}(x_w)$.

3. **Gaussian Label:** Generate a 2D Gaussian response centred at the patch centre:

$$y[r, c] = \exp\left(-\frac{(r - c_r)^2 + (c - c_c)^2}{2\sigma^2}\right), \quad \hat{Y} = \mathcal{F}_{\text{2D}}(y) \tag{3.2}$$

   with $\sigma = 2.0$ pixels and $(c_r, c_c) = (16, 16)$ for a centred $32 \times 32$ patch.

4. **Filter Computation:** Compute the frequency-domain filter coefficients:

$$\hat{\alpha}[k] = \frac{\hat{Y}[k]}{|\hat{X}[k]|^2 + \lambda}, \quad \lambda = 0.01 \tag{3.3}$$

The resulting $\hat{\alpha}$ array (1024 complex Q8.8 values, stored as pairs of 16-bit signed integers) is serialised to `data/alpha_hat.mem` using a Python script and loaded into an on-chip ROM at synthesis time via `$readmemh`.

Figure 3.1 and Figure 3.2 show the Hann window applied to a training patch and the corresponding Gaussian label used as the desired filter response.

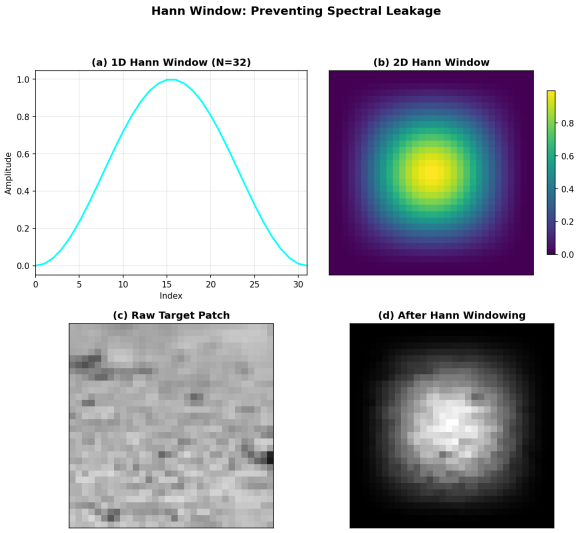



Figure 3.1: Hann window applied to a training patch. Spectral leakage is suppressed by tapering the patch edges to zero.

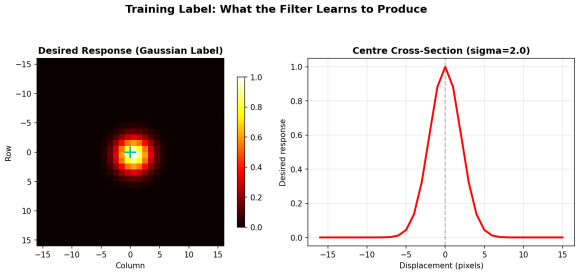



Figure 3.2: 2D Gaussian desired response ($\sigma$=2.0) used as the training label. The peak at centre encodes the target location.

### 3.1.2 Detection Phase

Given a new frame, a $32 \times 32$ search patch $z$ centred at the last known target location is extracted and processed:

1. Apply the 2D Hann window: $z_w = z \odot (w \otimes w)$
2. Compute $\hat{Z} = \mathcal{F}_{\text{2D}}(z_w)$
3. Compute the response map in the frequency domain:

$$\hat{R}[k] = \overline{\hat{X}[k]} \cdot \hat{Z}[k] \cdot \hat{\alpha}[k] \tag{3.4}$$

4. Compute the spatial response map: $R = \mathcal{F}_{\text{2D}}^{-1}(\hat{R})$
5. Locate the peak of $R$ — its $(row, col)$ offset from the patch centre gives the target displacement.

## 3.2 NCC – KCF Three-Frame Demo Pipeline

A software demonstration pipeline was developed in `scripts/kcf_demo_visual.py` implementing the handoff from NCC-based acquisition to KCF-based tracking:

- **Frame 1 (Training):** The user clicks to select the target centre in the reference image. A $32 \times 32$ patch is extracted, and KCF training is performed to compute $\hat{\alpha}$.
- **Frame 2 (NCC Acquisition):** NCC is computed between the template and a second real image. A valid-region mask ensures the search window stays within image boundaries. The peak of the normalised correlation map gives the initial target estimate in Frame 2.

- **Frame 3 (KCF Tracking):** A synthetic third frame is generated using `numpy.roll()` to introduce a controlled pixel-level shift. KCF detection (Equation 3.4) is run on the patch centred at the NCC result, and the response map peak provides the refined target position.

Figures 3.3–3.5 illustrate the three-stage pipeline on a real image pair.

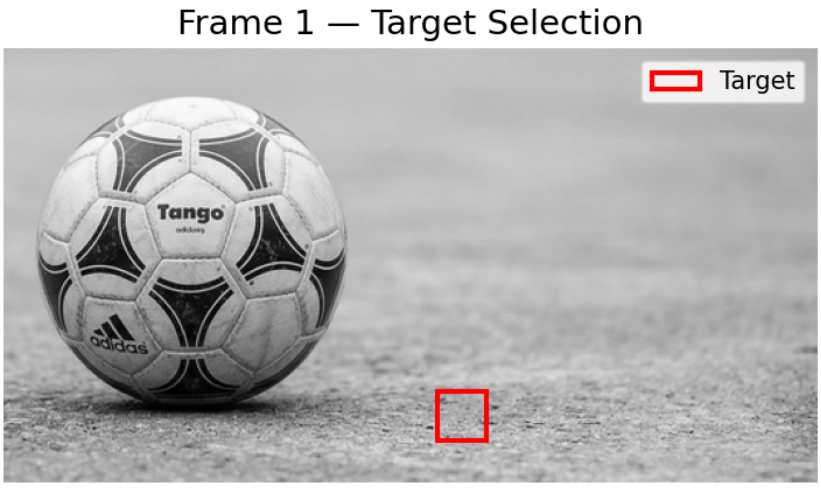



Figure 3.3: Frame 1: User-selected target region (green bounding box). The $32 \times 32$ patch centred here is used for KCF training.

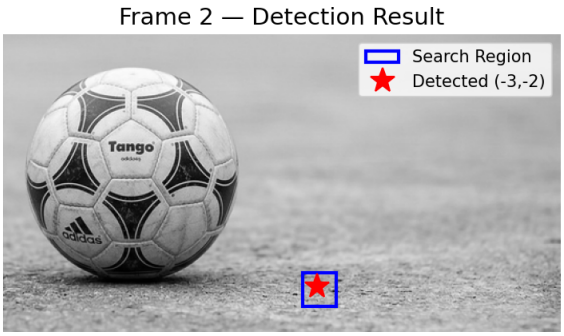



Figure 3.4: Frame 2: NCC search result. The heatmap shows correlation scores across the search region.

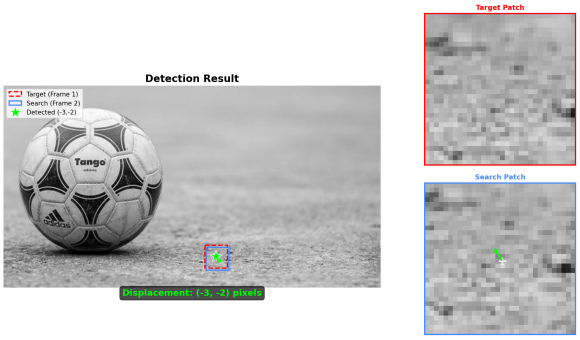



Figure 3.5: Frame 3: KCF detection result on the NCC-initialised patch. The bounding box marks the refined target position.

## 3.3 RTL Hardware Architecture

### 3.3.1 Top-Level FSM: `kcf_detect_top.sv`

The top-level RTL module `kcf_detect_top.sv` implements a 7-stage synchronous FSM that orchestrates the full KCF detection pipeline. Table 3.1 lists each stage with its approximate cycle count.

Table 3.1: KCF Detection FSM Pipeline Stages

| Stage | State Name | Cycles (approx.) | Operation |
|---|---|---|---|
| 0 | S_IDLE | — | Wait for start pulse |
| 1 | S_HANN_WIN | 1024 | Element-wise Hann multiply |
| 2 | S_FFT_LOAD | 32 | Load rows into FFT engine |
| 3 | S_FFT_RUN | ≈5600 | 2D-FFT (32 row + 32 col FFTs) |
| 4 | S_ELEM_MUL | 1024 | $\overline{\hat{X}} \cdot \hat{Z} \cdot \hat{\alpha}$ element-wise |
| 5 | S_IFFT_LOAD | 32 | Load into IFFT engine |
| 6 | S_IFFT_RUN | ≈5600 | 2D-IFFT (conjugate trick) |
| 7 | S_PEAK_RUN | 1024 | Scan for peak position |
| | S_DONE | 1 | Latch output, assert done |
| **Total** | | **≈19,300** | **≈ 193 $\mu$s @ 100 MHz** |

The done signal is a single-cycle pulse asserted in S_DONE. The outputs peak_row, peak_col, peak_val, and target_found are latched combinatorially and held valid until the next start pulse.

### 3.3.2 Building Blocks

The RTL design is composed of the following parameterised modules:

- butterfly.sv — Single butterfly unit computing $(a + W \cdot b,\ a - W \cdot b)$ using Q8.8 fixed-point complex multiplication.
- twiddle_rom_32.sv — Precomputed ROM of $W_N^k = e^{-j2\pi k/N}$ twiddle factors for $N = 32$.
- fft1d_32.sv — Iterative Radix-2 DIT FFT engine for 32-point 1D transforms. Processes one butterfly per cycle; completes in 80 cycles.
- fft2d_32.sv — Row-column 2D-FFT. Applies fft1d_32 to all 32 rows then all 32 columns.
- ifft2d_32.sv — 2D-IFFT implemented using the conjugate trick (Section 3.3.4).
- hann_rom_32.sv — ROM containing the 32-point Hann window coefficients in Q8.8.
- cconj_mul.sv — Complex conjugate multiplier: computes $\overline{A} \cdot B \cdot C$ for three Q8.8 complex operands.
- peak_finder.sv — Scans the $32 \times 32$ real-valued response map in 1024 cycles and outputs the $(row, col)$ of the maximum value.

### 3.3.3 2D-FFT Implementation

The 2D-FFT is computed via the separable row-column decomposition. For an input array $X[r][c]$ of size $N \times N$:

$$\mathcal{F}_{\text{2D}}(X) = \mathcal{F}_{\text{col}}(\mathcal{F}_{\text{row}}(X)) \tag{3.5}$$

The `fft2d_32` module first applies `fft1d_32` to each of the 32 rows (yielding a row-transformed buffer), then applies `fft1d_32` to each of the 32 columns of the intermediate result. Total cycle count is $2 \times 32 \times 80 = 5120$ cycles for the transform passes plus overhead for memory reads/writes.

### 3.3.4 IFFT via Conjugate Trick

Rather than implementing a separate IFFT datapath, the IFFT is computed by reusing the forward FFT engine with the following identity:

$$\mathcal{F}^{-1}(X) = \frac{\overline{\mathcal{F}(\bar{X})}}{N^2} \tag{3.6}$$

The implementation steps are: (1) conjugate the input array element-wise, (2) apply the forward 2D-FFT, (3) conjugate the result, (4) divide by $N^2 = 1024$ (implemented as an arithmetic right shift). This avoids duplicating the FFT butterfly logic and reduces LUT consumption.

## 3.4 Q8.8 Fixed-Point Arithmetic and Scaling

### 3.4.1 Fixed-Point Format

All arithmetic in the RTL pipeline uses the Q8.8 signed fixed-point format: 16-bit two's complement values with 8 integer bits and 8 fractional bits. One unit in the least significant position (ULP) represents $2^{-8} \approx 0.004$. The representable range is $[-128,\ +127.996]$.

The choice of Q8.8 is driven by the need to represent both the pixel values (range $[0, 255]$ after normalisation to $[0, 1]$, fitting in 8 fractional bits) and the FFT twiddle factors (range $[-1, +1]$, which require fractional bits).

### 3.4.2 Precision Problem with Naive Two-Pass Scaling

A standard 2D-FFT implementation applies a $1/N$ normalisation factor after each pass (row and column), resulting in a total $1/N^2 = 1/1024$ attenuation of the signal. In Q8.8, dividing all values by 1024 causes the response map peaks to round to zero, completely destroying the tracking output.

### 3.4.3 Scaling Solution: Asymmetric FFT and Adaptive Prescaling

Two complementary techniques are employed to solve the precision problem:

1. **Asymmetric FFT Scaling:** The $1/N$ normalisation is applied only in the row-pass of the FFT, not the column-pass. The column-pass is unscaled. This gives a total attenuation of $1/N$ instead of $1/N^2$, preserving 5 more bits of precision in the response map.

2. **Adaptive Filter Pre-scaling (constant $K$):** Before loading $\hat{\alpha}$ into the ROM, the Python training script computes a scale factor $K$ such that $\max|\hat{\alpha}| \times K < 120$ (just below the Q8.8 saturation limit of 127). All $\hat{\alpha}$ values are multiplied by $K$ before serialisation. The hardware response map is correspondingly scaled up by $K$, but since we only care about the *location* of the peak (not its absolute magnitude), this does not affect tracking accuracy.

Table 3.2 summarises all key algorithm parameters used in both the software reference model and the RTL implementation.

Table 3.2: KCF Algorithm and Hardware Parameters

| **Parameter** | **Value** | **Description** |
|---|---|---|
| $N$ | 32 | Patch side length (pixels) |
| `DATA_WIDTH` | 16 | Fixed-point word width (bits) |
| `FRAC` | 8 | Fractional bits (Q8.8) |
| $\lambda$ | 0.01 | Ridge regression regularisation constant |
| $\sigma$ | 2.0 | Gaussian label bandwidth (pixels) |
| `SCALE` | Adaptive $K$ | Filter pre-scale factor |
| `CONF_THRESHOLD` | Configurable | Minimum peak value for `target_found` |
| Clock frequency | 100 MHz | Target operating frequency |
| Total latency | ≈19,300 cycles | Per-patch detection time (≈193 $\mu$s) |

## 3.5 AXI Interface Design

### 3.5.1 Overview

To integrate the KCF IP core into a standard SoC environment, an AXI wrapper module `kcf_axi_wrapper.sv` is designed. It exposes two standard AMBA AXI interfaces:

- **AXI4-Stream Slave** (prefix `s_axis_`): receives a 32×32 patch as a stream of 1024 consecutive 16-bit pixels, with `TLAST` asserted on the final beat.
- **AXI4-Lite Slave** (prefix `s_axil_`): provides a memory-mapped register interface for control and result readout.

### 3.5.2 AXI4-Lite Register Map

Table 3.3 describes the six 32-bit registers exposed over AXI4-Lite (base address + offset).

Table 3.3: AXI4-Lite Register Map

| Offset | Register | Access | Description |
|---|---|---|---|
| 0x00 | CTRL | W | Bit 0: write 1 to trigger start pulse |
| 0x04 | THRESHOLD | R/W | Bits [15:0]: confidence threshold for target_found |
| 0x08 | PEAK_ROW | R | Bits [4:0]: row of response peak (0–31) |
| 0x0C | PEAK_COL | R | Bits [4:0]: column of response peak (0–31) |
| 0x10 | PEAK_VAL | R | Bits [15:0]: signed peak value (Q8.8) |
| 0x14 | TARGET_FOUND | R | Bit 0: 1 if peak_val > threshold |

### 3.5.3 SoC Integration Protocol

The standard software protocol for operating the KCF IP from a host processor is:

1. Write the confidence threshold to THRESHOLD register.
2. Initiate an AXI4-Stream DMA transfer of 1024 × 16-bit pixels from the frame buffer to the s_axis port. The IP's tready is asserted when the core is in the idle state.
3. Write 0x1 to the CTRL register to assert the start pulse after the patch has been fully received.
4. Poll or interrupt-wait on the PEAK_ROW register — alternatively, connect the done wire to a GPIO interrupt line.
5. Read PEAK_ROW, PEAK_COL, PEAK_VAL, and TARGET_FOUND.
6. Compute target displacement: $\Delta r = \text{PEAK\_ROW} - 16$, $\Delta c = \text{PEAK\_COL} - 16$.

This interface is compatible with any AXI4-capable processor including RISC-V (e.g., RV32I soft cores implemented on the same FPGA device) and ARM Cortex-M/A series processors connected via an AMBA AXI interconnect fabric.

## 3.6 Software Toolchain

The following Python scripts were developed as part of this project:

- **kcf_demo_visual.py** — Interactive three-frame demo (NCC acquisition to KCF detection) with annotated OpenCV visualisation, demonstrating the full pipeline on real image pairs.
- **gen_data_32.py** — Generates alpha_hat.mem, test_patch_32.mem, and test2_patch_32.mem in hexadecimal format for Vivado/Icarus simulation via $readmemh.

## 3.7 Verification and Results

### 3.7.1 RTL Simulation

The RTL design was simulated using Icarus Verilog. The testbench loads pre-computed patches and the reference filter from the `data/` directory, drives the `start` signal, and monitors the `done` pulse and peak output registers.

Figure 3.6 shows the Vivado simulation waveform, confirming that the `done` signal is asserted after approximately 19,300 cycles and the peak coordinates are correctly latched.

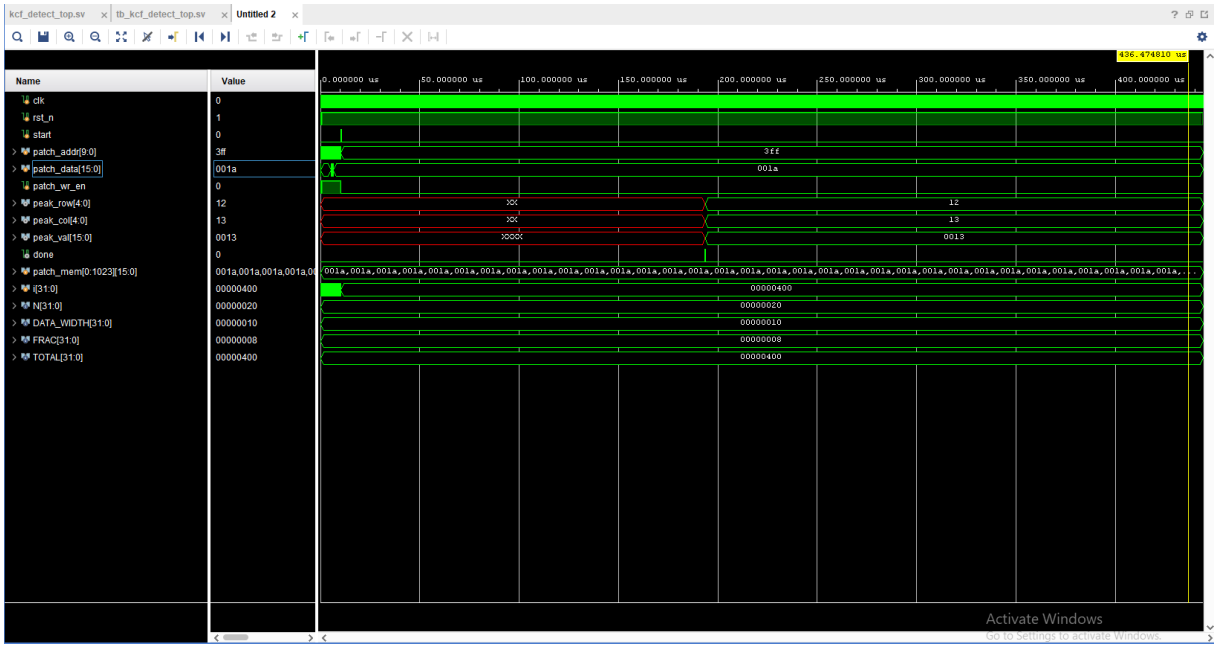



Figure 3.6: Vivado simulation waveform showing the `done` pulse and output peak coordinates after the full KCF detection pipeline completes.

### 3.7.2 Hardware–Software Agreement

The peak $(row, col)$ coordinates output by the RTL simulation were compared against the Python floating-point reference model for both test patches. Figure 3.7 shows the response maps from both implementations alongside the detected peak locations.

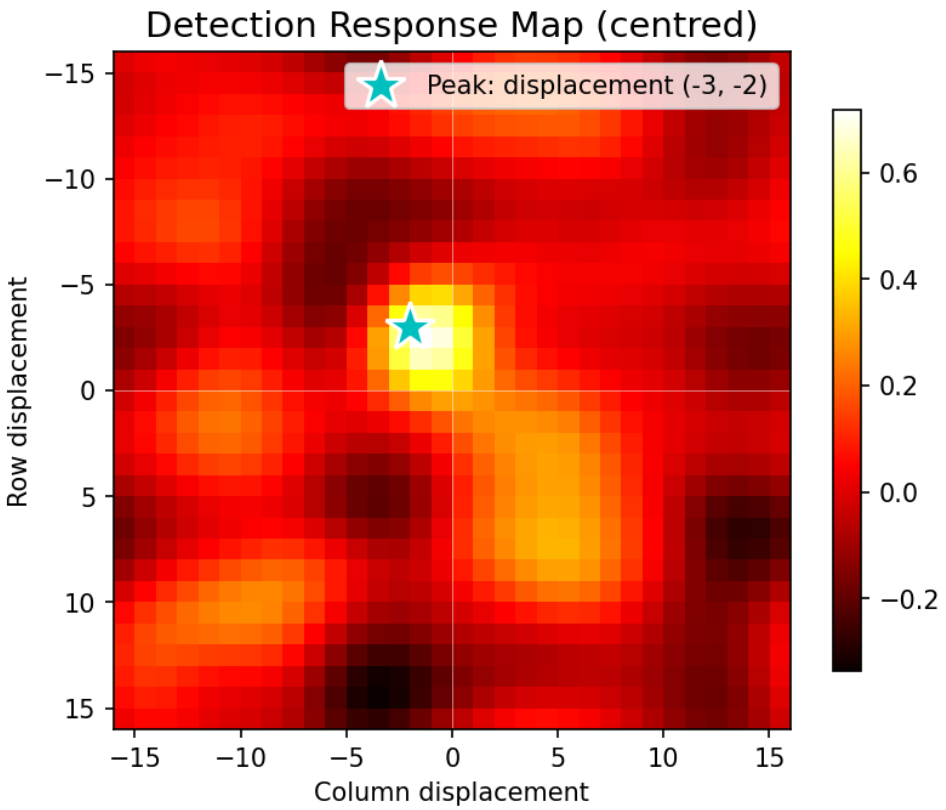



Figure 3.7: KCF response map (2D heat-map) from the Python reference model. The peak at $(row{=}18, col{=}19)$ matches the RTL simulation output, confirming hardware–software agreement.

# Chapter 4

# Conclusion

This project presents a complete, synthesisable RTL implementation of the Kernelized Correlation Filter visual tracking algorithm in SystemVerilog. The design successfully demonstrates all core stages of the KCF detection pipeline:

- A 7-stage FSM orchestrating Hann windowing, 2D-FFT, element-wise complex correlation, 2D-IFFT, and peak detection.
- A novel asymmetric FFT scaling scheme combined with adaptive filter pre-scaling that resolves the precision loss inherent to two-pass Q8.8 fixed-point FFT normalisation.
- The IFFT conjugate trick, which eliminates the need for a separate IFFT datapath by reusing the forward FFT hardware.
- An AXI4-Stream and AXI4-Lite wrapper providing a standard SoC-compatible interface.
- A Python software toolchain providing a floating-point reference model, test vector generation, and an interactive tracking demonstration.

Hardware–software agreement was verified between the Vivado RTL simulation and the Python reference model for both test patches. The full pipeline executes in approximately 19,300 clock cycles, corresponding to a detection latency of $\approx$193 $\mu$s at a 100 MHz clock, which is well within real-time video processing budgets for 30 fps ($\approx$33 ms per frame).

The design is validated at the register-transfer level and is ready for FPGA synthesis on the Xilinx Artix-7 (Nexys A7-100T) device using Vivado 2024.2. The AXI wrapper exposes the core as a plug-and-play IP block compatible with any AXI4-capable processor, positioning the design as a practical building block for embedded vision systems.

# Chapter 5

# Future Prospects

Several extensions are planned or recommended to evolve this design toward a production-ready embedded tracking IP:

- **FPGA Synthesis and Timing Closure:** Synthesise the current RTL on the Artix-7 XC7A100T device and report LUT, flip-flop, BRAM, and DSP48 utilisation. Close timing at 100 MHz and evaluate the maximum achievable frequency.
- **SoC Integration via AXI Interconnect:** Connect the KCF IP core to a RISC-V or ARM soft processor on the same FPGA device using the AXI4-Lite and AXI4-Stream interfaces designed in Chapter 3. This enables a complete embedded tracking system with host control.
- **Higher Resolution Support (64×64):** Extend the FFT engine to support 64×64 patches (6-stage Radix-2 DIT) to improve tracking accuracy on larger targets and enable operation at standard video resolutions.
- **Gaussian Kernel Support:** Implement the full Gaussian (RBF) kernel by adding a hardware exponential Look-Up Table (LUT) for $e^x$. This extends the feature space and improves robustness to appearance changes.
- **Online Model Update:** Add a hardware update path that combines the current filter with a new one via a learning rate $\eta$: $\hat{\alpha}_{\text{new}} = (1-\eta)\hat{\alpha}_{\text{old}} + \eta\hat{\alpha}_{\text{frame}}$. This allows the tracker to adapt to gradual appearance changes without host CPU intervention.
- **Camera / HDMI Input Pipeline:** Integrate a camera input module (MIPI CSI-2 or OV7670 parallel) and HDMI output module to create a fully self-contained real-time tracking demonstration system on the Nexys A7 board.

# Bibliography

[1] J. F. Henriques, R. Caseiro, P. Martins, and J. Batista, "High-speed tracking with kernelized correlation filters," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 37, no. 3, pp. 583–596, 2015.

[2] D. S. Bolme, J. R. Beveridge, B. A. Draper, and Y. M. Lui, "Visual object tracking using adaptive correlation filters," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2010, pp. 2544–2550.

[3] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.

[4] M. Danelljan, G. Häger, F. S. Khan, and M. Felsberg, "Accurate scale estimation for robust visual tracking," in *British Machine Vision Conference (BMVC)*. BMVA Press, 2014.